KAYAK MERKEZİ OTOMASYONU

AlpineOS

Üniversite projemin yeniden tasarlanmış web uygulaması

Fatih Yılmaz

Bu projeyi neden yeniden geliştirdim?

İlk tasarımdaki veritabanı fikrini gerçek uygulama akışına taşımak istedim.

İLK ÇALIŞMA

Müşteri, ekipman, kiralama ve konaklama için ilişkisel veritabanı tasarımı.

GÖRDÜĞÜM İHTİYAÇ

İşlemler tek bir yerde yürüsün; kayıtlar ve kaynak uygunluğu tutarlı kalsın.

YENİ YAKLAŞIM

İki kullanıcı paneli, ortak API, rol denetimi ve PostgreSQL iş kuralları.

Çözmek istediğim problem

- Ayrı yürütülen kiralama, konaklama ve personel kayıtları
- Aynı ekipman veya odanın çakışan rezervasyonlarla kullanılması
- Müşterinin kendi rezervasyonlarını kolayca takip edememesi
- İşletme yöneticisinin güncel operasyon bilgisine ulaşma ihtiyacı

Sistemin iki kullanıcısı var

Her iki panel de aynı API ve PostgreSQL veritabanını kullanıyor.

YÖNETİCİ VE PERSONEL

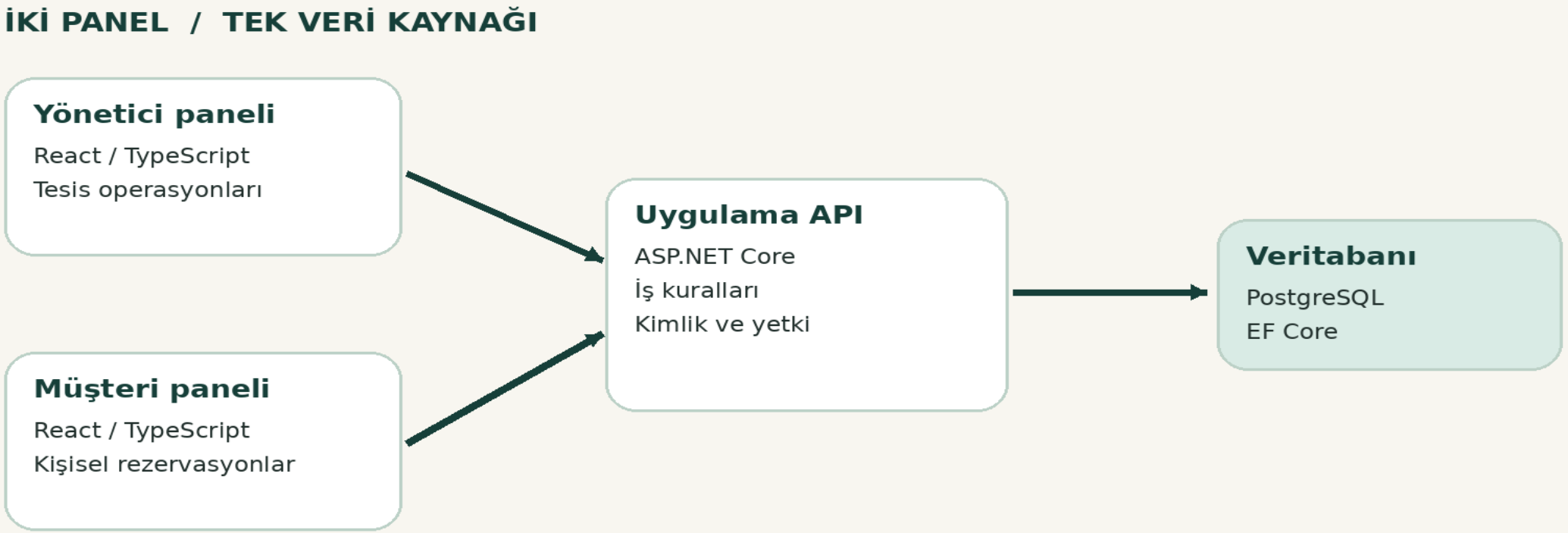
Kiralama, envanter, oda, müşteri, pist, etkinlik, finans ve personel yönetimi.

MÜŞTERİ

Katalog inceleme, rezervasyon oluşturma, derslere kayıt, kendi işlemlerini ve profilini takip etme.

Teknik mimarim

Web arayüzünden gelen talepleri tek servis üzerinden işledim.



Teknoloji seçimlerim

C# / ASP.NET Core

API ve iş kuralları

React / TypeScript

İki panelin arayüzü

PostgreSQL / EF Core

Veri ve ilişkiler

Docker Compose / Nginx

Yerel dağıtım

Veritabanını nasıl kurguladım?

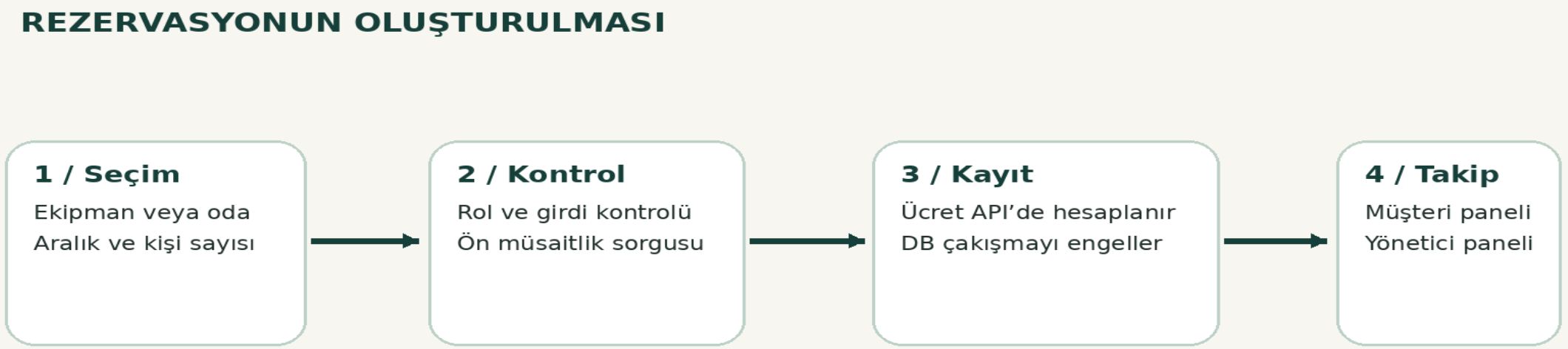
Kayıtlar birbirine ait, fakat işlem türleri birbirinden ayrılmış durumda.

BİRİBİRİYLE İLİŞKİLİ ANA ALANLAR



Rezervasyon akışı

En kritik iş kuralı: aynı kaynak aynı anda iki rezervasyona verilemez.



Yönetici panelinde neler var?

- Kontrol paneli: göstergeler, finans ve tesis özeti
- Ekipman envanteri, kiralama, teslim ve iade işlemleri
- Odalar, konaklama, pistler ve ders / etkinlik kayıtları
- Müşteri, personel, departman, rol ve finans yönetimi

Müşteri panelini ayrıca geliştirdim

- E-posta ve parola ile müşteri hesabı
- Ekipman ve konaklama seçeneklerini inceleme
- Müsaitlik kontrolü ile rezervasyon oluşturma
- Rezervasyonlarım, ödeme geçmişim, profilim ve pist durumu

Rol ayrımı ve güvenlik

PERSONEL

Admin, Reception, Equipment, Slope ve Cashier rolleri; yetkili işletme işlemleri.

MÜŞTERİ

Oturumuna ait profil ve rezervasyonlar. Başka müşterinin kayıtlarına erişim yok.

Şifre hashleme, cookie oturumu ve CSRF kontrolü ek koruma katmanlarıdır.

Demo verilerle sistemi gösterebiliyorum

Kayıtlar kurgu amaçlıdır; gerçek işletme verisi değildir.

48
Ekipman

18
Konaklama

12
Pist

16
Etkinlik

9
Departman

16
Çalışan

Çalışan uygulamanın ekran görüntüleri

Bu slaytı gerçek ekran görüntülerimi ekledikten sonra tamamlayacağım.

Görsel buraya eklenecek

YÖNETİCİ KONTROL PANELİ

Görsel buraya eklenecek

MÜŞTERİ KEŞFET EKRANI

Bir rezervasyonun iki panelde görünmesi

MÜŞTERİ TARAFI

Ekipman seçimi → uygunluk kontrolü → rezervasyon → Rezervasyonlarım

YÖNETİCİ TARAFI

Aynı kaydı görme → teslim / iade → durum takibi

Test yaklaşımım ve sınırlar

- Uygulamayı yerel Docker ortamında çalıştırdım.
- Müşteri panelinde karşılaşılan TypeScript tip sorununu düzelttim.
- Otomatik test dosyaları var; tüm testlerin geçtiği iddia edilmiyor.
- Gerçek online tahsilat, faturalama ve üretim güvenliği henüz kapsam dışı.

Bu projeden neler öğrendim?

- Veritabanındaki ilişkiyi iş akışına dönüştürmek
- API doğrulaması ve veri bütünlüğünü birlikte düşünmek
- İki farklı kullanıcı rolüne uygun arayüz tasarlamak
- Kod, veritabanı, Docker ve dokümantasyonu bir arada yönetmek

SONUÇ

AlpineOS

Veritabanı projesinden iki panelli bir web uygulamasına.

Fatih Yılmaz / Kayak Merkezi Otomasyonu